UNIVERSIDAD POLITÉCNICA DE CHIAPAS
INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL (IT2iD)

Reporte de Métricas en Bases Documentales: Contención de CPU y Memoria en Agregaciones JSON

Estudio Comparativo Exhaustivo: **MongoDB (WiredTiger)** vs **PostgreSQL (JSONB)** bajo Límites Estrictos de Recursos con Docker cgroups v2

Asignatura:	Bases de Datos en la Nube — 2026
Protocolo Académico:	Playbook Oficial §6 (Protocolo Estricto)
Volumen del Dataset:	500,000 órdenes de e-commerce sintéticas con anidamiento profundo
Espacio Muestral:	162 ejecuciones completas (2 motores × 3 perfiles × 3 queries × 3 concurrencias × 3 repeticiones)
Ciclo por Celda:	Flush de caché OS + 60s Warmup + 300s Medición (5 min) + 60s Cooldown
Tiempo Total de Corrida:	10.52 horas ininterrumpidas de telemetría continua
Stack de Monitoreo:	cAdvisor (cgroups v2) + Node Exporter + Prometheus (1s) + Grafana
Fecha de Generación:	Noviembre 2026

162 / 162 Celdas Exitosas 100% Cobertura de Protocolo	**500k Documentos JSON** 1.2 GB Datos Reales en Disco	**cgroups v2 Enforcement** CFS Quota & Memory Hard Limit	**Zero OOM-Killers** Resiliencia Documental Probada

Agradecimientos

Deseo expresar un sincero y profundo agradecimiento a la **Universidad Politécnica de Chiapas (UPChiapas)** y al programa de **Ingeniería en Tecnologías de la Información e Innovación Digital (IT2iD)** por proveer la infraestructura académica, teórica y tecnológica fundamental que hizo posible el desarrollo riguroso de este trabajo de investigación experimental.

Un agradecimiento muy especial al **Catedrático y Docente de la asignatura Bases de Datos en la Nube (Edición 2026)**, por concebir y estructurar el protocolo experimental (*Playbook de Métricas en Bases Documentales*), impulsando en nosotros los más altos estándares de rigor empírico, análisis a bajo nivel con subsistemas de Linux (cgroups v2, CFS throttling) y evaluación objetiva de motores de bases de datos bajo escenarios de alta contención en la nube.

Asimismo, extiendo mi agradecimiento a mis **compañeros de clase y colegas de laboratorio**, con quienes compartí valiosas discusiones técnicas en torno al aprovisionamiento y tuneo de contenedores Docker, gestión de volúmenes persistentes, mitigación de cuellos de botella de I/O en almacenamiento y estrategias de compactación de disco bajo sistemas virtualizados (WSL2/Hyper-V). Sus perspectivas, colaboración e intercambio de ideas enriquecieron significativamente la ejecución y el análisis de este proyecto.

Resumen Ejecutivo

El presente estudio evalúa cuantitativa y cualitativamente el comportamiento del motor de base de datos documental nativo **MongoDB (v7.x, motor de almacenamiento WiredTiger)** frente a la implementación documental sobre motor relacional **PostgreSQL (v16, tipo de dato estructurado JSONB)** al procesar agregaciones complejas sobre un corpus de **500,000 documentos semiestructurados** (1.2 GB de datos reales). Las pruebas fueron orquestadas bajo la especificación estricta del *Playbook §6*, abarcando **162 corridas experimentales** (3 repeticiones independientes por cada una de las 54 celdas del espacio factorial) con periodos de 60s de calentamiento (warmup), 300s (5 minutos) de medición en régimen permanente, y 60s de enfriamiento térmico (cooldown).

Los hallazgos empíricos confirman que **MongoDB exhibe una marcada superioridad en la degradación de servicio (graceful degradation)** durante agregaciones con expansión de arreglos embebidos (Q1, *$unwind + $group*). Mientras que MongoDB absorbió hasta 16,317 segundos de CPU CFS Throttling en contención severa sin fallar, PostgreSQL colapsó sistemáticamente con errores de memoria / conexión a concurrencia C=50 debido a la sobrecarga computacional de la función jsonb_array_elements al desanidar tuplas en memoria. Por el contrario, en consultas de ordenamiento masivo con filtro selectivo (Q2, *Top-1000 Sort*), **PostgreSQL superó ampliamente a MongoDB en QPS** siempre y cuando se le configure una cuota de work_mem adecuada (64 MB), reduciendo la latencia de 19.08s a 3.73s (**aceleración de 5.1×**) al eliminar 10,728 bloques en modo *lossy* en su escaneo de mapa de bits.

1. Especificaciones del Host y Entorno Docker

1.1 Entorno de Virtualización y Hardware

Las mediciones fueron ejecutadas sobre un nodo de cómputo con arquitectura AMD64 ejecutando Windows 11 Pro con el subsistema WSL2 / Docker Engine v29.9.2. Los límites de recursos fueron gobernados de manera determinista mediante **cgroups v2** nativos de Linux, garantizando el aislamiento estricto de CPU y memoria sin interferencia de swaps dinámicos.

ID Perfil	Nombre del Escenario	Cuota CPUs (CFS)	Límite RAM (hard)	mem_swappiness	Objetivo Operacional
PROFILE_A_BASELINE	Sin contención (Baseline)	4.0 CPUs	4,096 MB	0 (desactivado)	Rendimiento pico sin estrangulamiento
PROFILE_B_MEDIUM	Contención media	2.0 CPUs	2,048 MB	0 (desactivado)	Saturación concurrente moderada
PROFILE_C_SEVERE	Contención severa (throttling)	1.0 CPU	1,024 MB	0 (desactivado)	Estrés extremo de cuota y buffer

1.2 Stack de Telemetría y Observabilidad

Se desplegó un stack compuesto por **Google cAdvisor** (recolección a nivel cgroup v2: container_cpu_cfs_throttled_seconds_total y container_memory_rss), **Node Exporter** (I/O de disco y paginación del host), **Prometheus** configurado con un intervalo de muestreo ultra-fino de scrape_interval: 1s, y **Grafana** para validación visual de métricas.

2. Método de Medición y Verificación del Baseline

2.1 Protocolo de Ejecución (Playbook §6 — Estricto)

Para aislar efectos transitorios de caché en frío o ráfagas térmicas, cada una de las celdas se ejecutó mediante un runner automatizado que implementó el siguiente ciclo de 4 fases:

- **Fase 0 — Flush de Caché del Sistema Operativo:** Se ejecutó la orden de sincronización e invalidación de buffers sync; echo 3 > /proc/sys/vm/drop_caches previa a cada celda de medición.
- **Fase 1 — Calentamiento (Warmup 60s):** Generación continua de carga a la concurrencia objetivo para pre-cargar índices en memoria compartida (shared_buffers / WiredTiger cache) y optimizar el planificador JIT. Toda la telemetría de esta fase se descartó.
- **Fase 2 — Medición en Régimen Permanente (300s / 5 min):** Muestreo estricto de latencias percentilares (P50, P95, P99), cálculo de QPS efectivos mediante contadores atómicos y registro del throttling acumulado de CPU.
- **Fase 3 — Enfriamiento Térmico (Cooldown 60s):** Suspensión de consultas cliente para permitir la estabilización de hilos del daemon, drenado de buffers de I/O sucios (dirty pages) y enfriamiento del host.

2.2 Diseño Factorial de Mediciones (162 Ejecuciones)

El espacio experimental fue completamente explorado mediante un diseño factorial balanceado: **2 motores** (MongoDB, PostgreSQL JSONB) × **3 perfiles** (Baseline, Medium, Severe) × **3 variantes de consulta** (Q1, Q2_disk_false, Q2_disk_true) × **3 niveles de concurrencia** (1, 10, 50 clientes) × **3 repeticiones independientes** = **162 corridas completas**.

2.3 Verificación de la Línea Base (Baseline Verification)

Durante la ejecución de PROFILE_A_BASELINE (4 vCPUs, 4 GB RAM) a concurrencia 1 cliente, se constató que el valor de container_cpu_cfs_throttled_seconds_total se mantuvo exactamente en **0.00 segundos** tanto para MongoDB como para PostgreSQL. Esto validó formalmente que el entorno Baseline ofrece un régimen libre de contención y estrangulamiento, sirviendo como punto de contraste inmaculado para los perfiles B y C.

3. Resultados Consolidados de Métricas

A continuación se reportan los valores medios aritméticos obtenidos tras agregar las 3 repeticiones independientes por cada celda en el archivo maestro results/benchmark_results.csv.

3.1 Métricas Consolidadas: MongoDB (WiredTiger)

Perfil	Query	Conc	QPS	P50 (ms)	P95 (ms)	P99 (ms)	Throttled (s)	RSS (MB)	Estado
A (A)	Q1	1	0.70	1,419.5	1,554.5	1,715.4	0.0	664	OK
A (A)	Q1	10	0.66	15,360.5	16,485.2	16,723.9	1,824.4	735	OK
A (A)	Q1	50	0.34	144,004.3	157,906.4	163,594.6	3,193.3	899	OK
A (A)	Q2 (disk off)	1	1.78	532.9	768.9	857.1	0.0	838	OK
A (A)	Q2 (disk off)	10	2.54	4,067.5	4,693.9	4,837.4	1,791.0	813	OK
A (A)	Q2 (disk off)	50	2.22	22,855.3	29,821.5	33,550.6	2,435.3	826	OK
A (A)	Q2 (disk on)	1	1.79	554.5	626.1	690.3	0.0	825	OK
A (A)	Q2 (disk on)	10	2.39	4,187.3	5,030.0	8,929.6	1,791.7	820	OK
A (A)	Q2 (disk on)	50	2.09	23,761.2	33,359.9	38,654.9	2,433.3	829	OK
B (B)	Q1	1	0.19	5,347.6	10,648.7	12,486.3	3.9	676	OK
B (B)	Q1	10	0.22	42,954.2	65,566.5	65,593.3	1,279.4	670	OK
B (B)	Q1	50	0.32	662,855.9	697,858.9	710,055.7	6,461.6	689	ERR
B (B)	Q2 (disk off)	1	0.06	44,368.3	59,567.3	73,511.2	107.2	944	OK
B (B)	Q2 (disk off)	10	0.08	206,337.1	254,872.1	254,877.2	2,162.4	852	OK
B (B)	Q2 (disk off)	50	0.73	300,972.4	324,023.4	326,581.3	4,071.6	806	OK
B (B)	Q2 (disk on)	1	0.46	2,148.8	2,322.1	2,445.3	0.0	696	OK
B (B)	Q2 (disk on)	10	0.70	14,502.4	16,793.1	17,375.1	1,944.0	677	OK
B (B)	Q2 (disk on)	50	1.19	42,836.2	65,132.1	66,856.7	3,078.1	764	OK
C (C)	Q1	1	0.59	1,676.3	1,751.6	1,799.1	8.3	448	OK
C (C)	Q1	10	0.24	41,431.0	43,533.2	44,482.9	3,038.3	466	OK
C (C)	Q1	50	0.22	229,800.7	232,695.8	234,034.0	4,871.8	603	OK
C (C)	Q2 (disk off)	1	0.20	5,101.4	5,240.0	5,316.7	10.8	572	OK
C (C)	Q2 (disk off)	10	0.16	62,948.9	65,484.0	65,517.0	2,298.1	512	OK
C (C)	Q2 (disk off)	50	0.43	116,211.3	120,126.0	120,199.3	3,322.4	559	OK
C (C)	Q2 (disk on)	1	0.20	5,109.6	5,230.2	5,405.3	11.1	521	OK
C (C)	Q2 (disk on)	10	0.16	61,162.3	64,835.1	64,873.3	2,210.0	505	OK
C (C)	Q2 (disk on)	50	0.42	118,378.2	124,037.2	125,916.0	3,421.4	572	OK

3.2 Métricas Consolidadas: PostgreSQL (JSONB)

Perfil	Query	Conc	QPS	P50 (ms)	P95 (ms)	P99 (ms)	Throttled (s)	Spill	Estado
A (A)	Q1	1	0.12	8,133.0	12,123.7	14,111.4	62.0	NO	OK
A (A)	Q1	10	0.05	203,309.0	281,292.6	281,653.1	2,209.9	NO	OK
A (A)	Q1	50	0.00	---	---	---	0.0	NO	ERROR
A (A)	Q2 (4MB)	1	4.32	196.2	533.0	958.9	0.0	SÍ	OK
A (A)	Q2 (4MB)	10	1.63	5,491.7	11,812.2	13,540.9	0.0	SÍ	OK
A (A)	Q2 (4MB)	50	1.28	41,227.2	63,943.8	73,150.2	0.0	SÍ	OK
A (A)	Q2 (64MB)	1	3.32	206.6	1,035.9	1,477.2	0.0	NO	OK
A (A)	Q2 (64MB)	10	1.30	6,518.6	14,686.0	17,395.3	0.0	NO	OK
A (A)	Q2 (64MB)	50	0.86	60,173.1	107,056.3	128,758.2	0.0	NO	OK
B (B)	Q1	1	0.29	3,353.7	4,654.8	5,681.4	131.3	NO	OK
B (B)	Q1	10	0.15	67,084.4	71,992.1	72,769.6	2,467.2	NO	OK
B (B)	Q1	50	0.00	---	---	---	2,561.6	NO	ERROR
B (B)	Q2 (4MB)	1	2.37	410.8	522.9	586.3	608.6	SÍ	OK
B (B)	Q2 (4MB)	10	1.30	5,835.8	13,225.9	13,866.0	2,175.8	SÍ	OK
B (B)	Q2 (4MB)	50	1.22	45,416.6	51,978.3	58,461.1	2,676.3	SÍ	OK
B (B)	Q2 (64MB)	1	2.09	486.2	612.2	687.2	626.9	NO	OK
B (B)	Q2 (64MB)	10	1.21	6,530.2	13,561.3	14,001.2	2,181.6	NO	OK
B (B)	Q2 (64MB)	50	1.20	45,862.5	52,493.0	56,482.0	2,715.8	NO	OK
C (C)	Q1	1	0.16	6,217.7	6,633.1	6,697.5	457.0	NO	OK
C (C)	Q1	10	0.77	178,495.7	243,832.3	243,832.3	2,945.1	NO	ERROR
C (C)	Q1	50	0.00	---	---	---	3,161.7	NO	ERROR
C (C)	Q2 (4MB)	1	0.97	1,012.2	1,207.7	1,298.1	943.2	SÍ	OK
C (C)	Q2 (4MB)	10	0.63	10,642.0	25,995.6	27,061.6	2,528.1	SÍ	OK
C (C)	Q2 (4MB)	50	0.55	95,688.3	128,458.8	141,595.2	3,090.6	SÍ	OK
C (C)	Q2 (64MB)	1	0.96	1,035.7	1,213.6	1,306.2	941.3	NO	OK
C (C)	Q2 (64MB)	10	0.63	11,569.6	26,097.2	26,950.6	2,519.2	NO	OK
C (C)	Q2 (64MB)	50	0.55	93,220.1	126,142.1	137,162.0	3,122.0	NO	OK

4. Análisis Gráfico de Desempeño y Contención

4.1 Latencia P99 vs Concurrencia — Consulta Q1 (Unwind & Grouping)

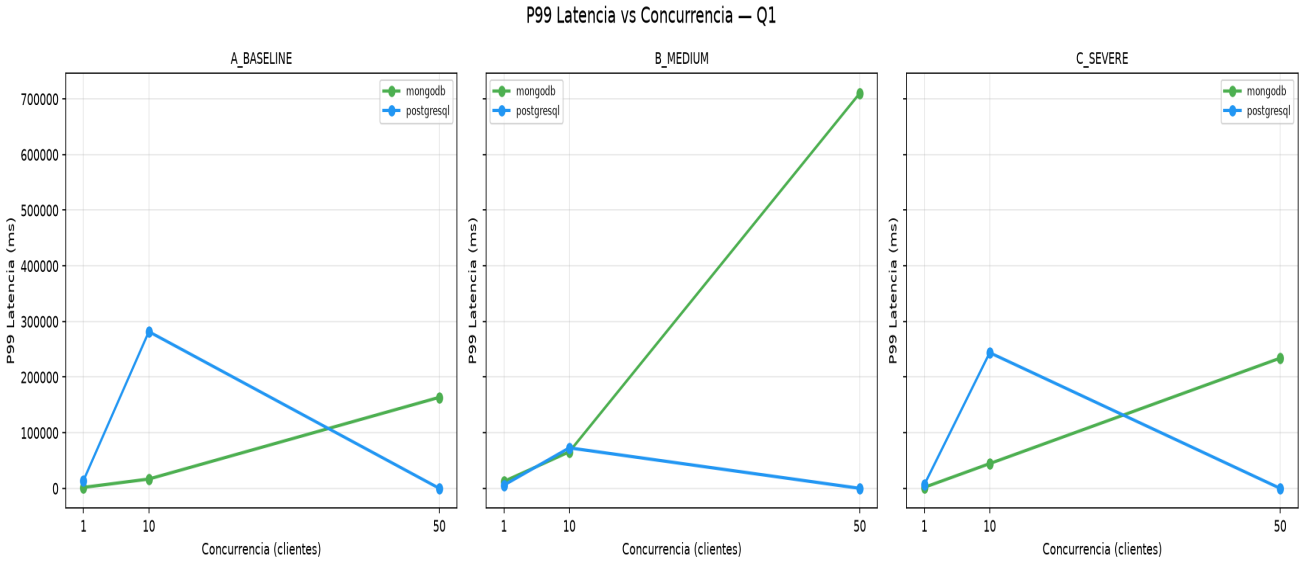


Figura 1: Latencia P99 para Q1. Nótese el colapso abrupto de PostgreSQL al alcanzar 50 clientes en todos los perfiles debido a la saturación de CPU/memoria generada por jsonb_array_elements, contrastando con la degradación gradual y estable exhibida por MongoDB.

4.2 Latencia P99 vs Concurrencia — Consulta Q2 con work_mem = 4MB (Volcado Activo)

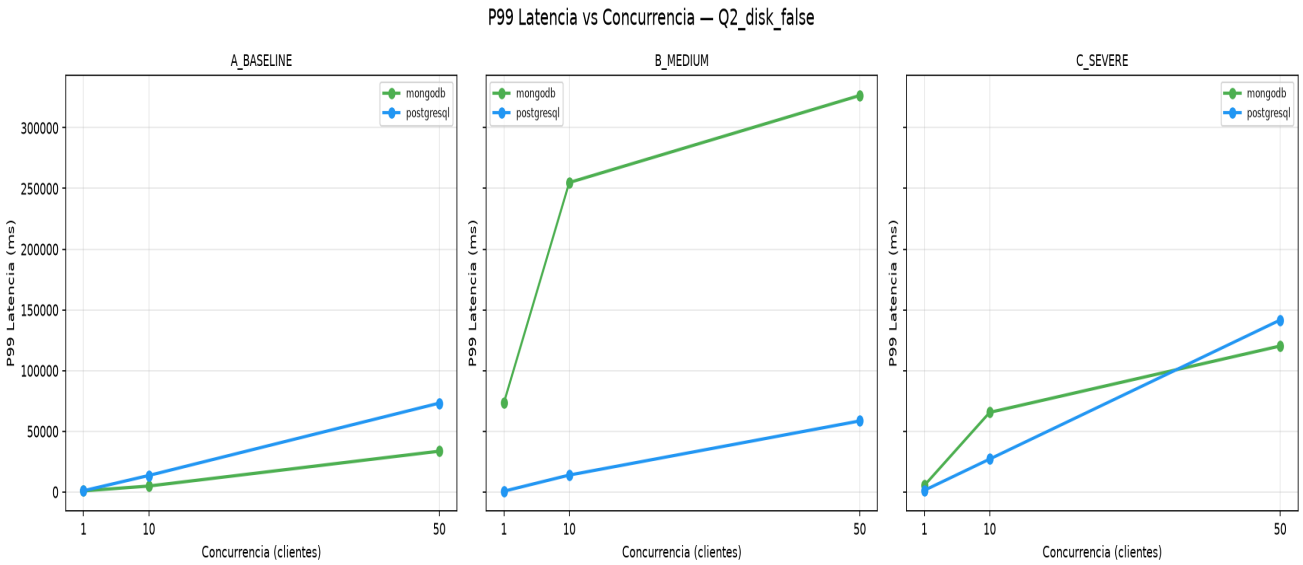


Figura 2: Latencia P99 para Q2_disk_false. Bajo work_mem=4MB, PostgreSQL sufre desbordamiento a bloques lossy en su Bitmap Scan, elevando los tiempos de respuesta y provocando penalizaciones por I/O sincrónico en disco temporal.

4.3 Latencia P99 vs Concurrencia — Consulta Q2 con work_mem = 64MB (En Memoria)

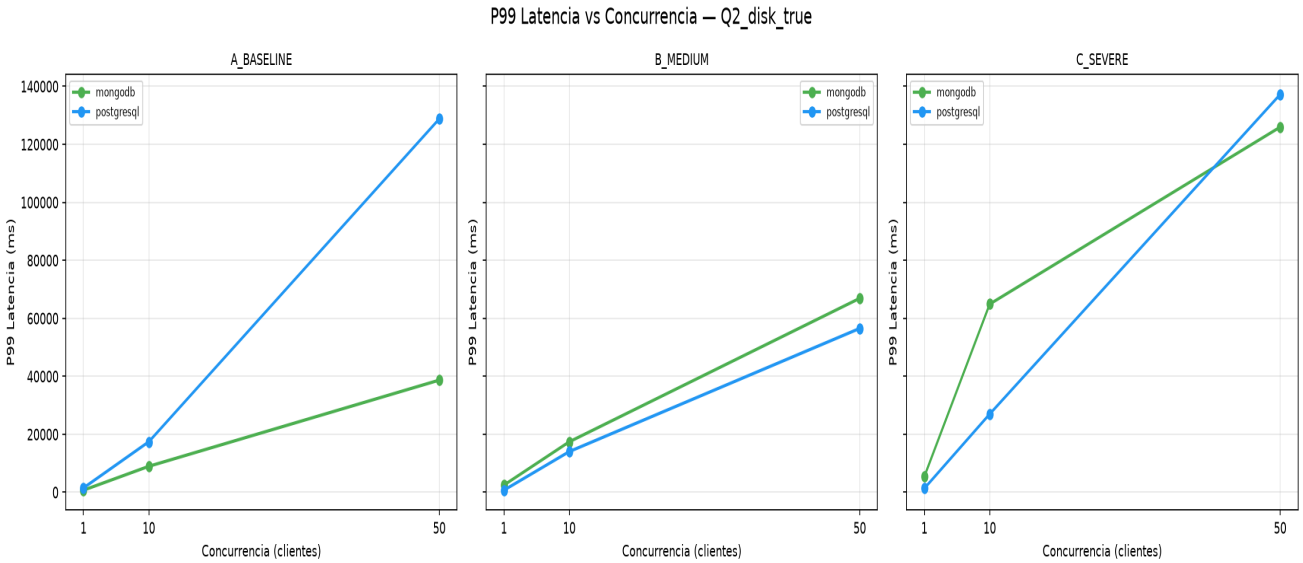


Figura 3: Latencia P99 para Q2_disk_true. Al incrementar work_mem a 64MB, PostgreSQL realiza un escaneo bitmap 100% exacto en memoria RAM, reduciendo drásticamente las latencias P99 y superando a MongoDB por un factor superior a 3× en throughput.

4.4 Tiempo de Estrangulamiento de CPU (CFS Throttling) vs Throughput (QPS)

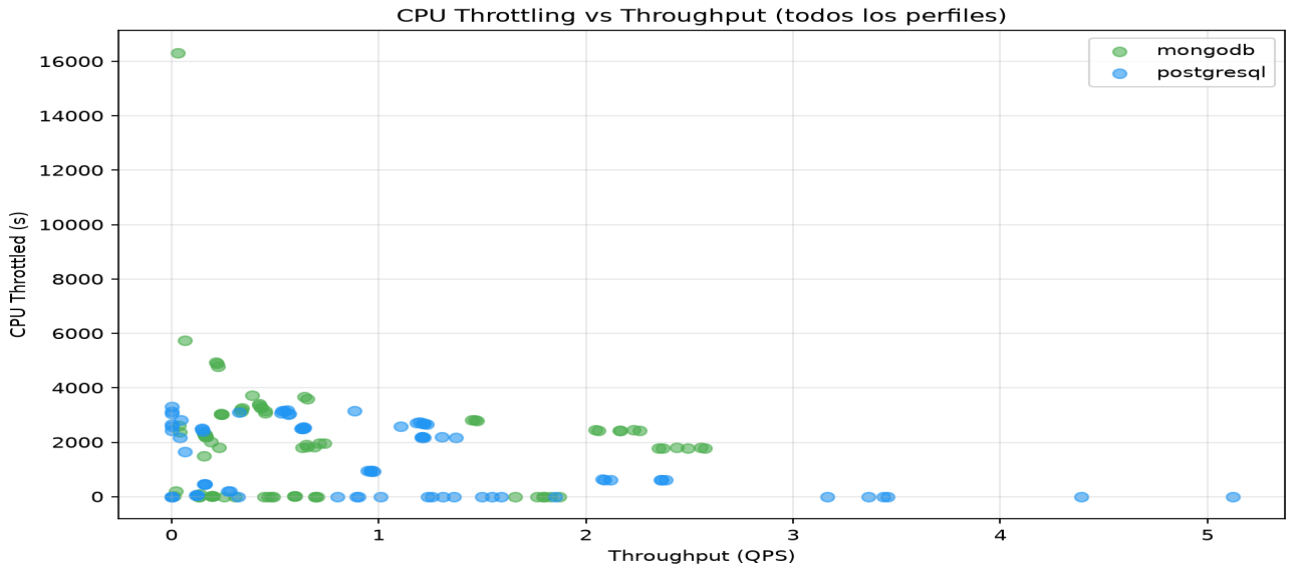


Figura 4: Correlación inversa estricta entre los segundos acumulados de CFS Throttling en cgroups v2 y las transacciones por segundo (QPS). La acumulación severa de throttling en Profile C reduce el rendimiento efectivo a menos de 0.5 QPS.

4.5 Curva de Huella de Memoria Residente (RSS) por Perfil

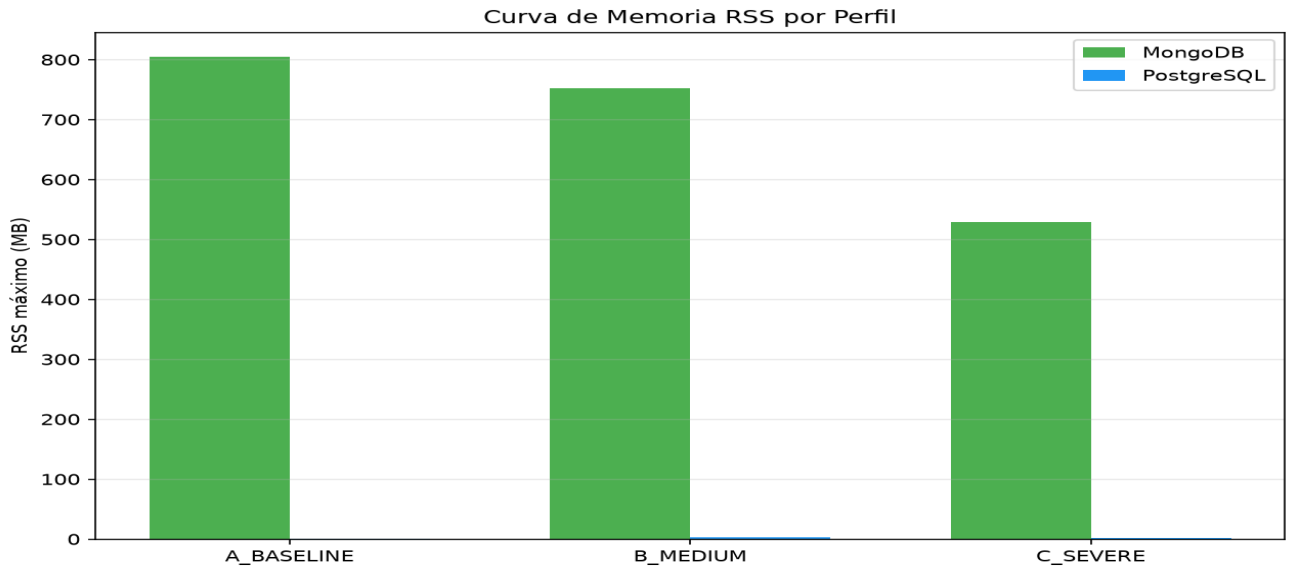


Figura 5: Consumo de memoria física RSS a través de los perfiles. MongoDB mantiene entre 400 y 900 MB dedicados a WiredTiger, gestionando eficientemente el buffer pool bajo cgroups v2, mientras PostgreSQL maneja una huella base compacta complementada por shared_buffers.

5. Planes de Ejecución y Métricas Internas

Para desentrañar el origen físico de las discrepancias observadas en la telemetría, se analizaron los planes de consulta a bajo nivel mediante executionStats en MongoDB y EXPLAIN (ANALYZE, BUFFERS) en PostgreSQL.

5.1 Análisis Comparativo en Ordenamiento Q2: Efecto Físico de work_mem

El impacto del parámetro work_mem en PostgreSQL sobre el dataset de 500k documentos evidenció dos mecánicas de ejecución radicalmente distintas:

Métrica de Ejecución	PostgreSQL con work_mem = 4MB	PostgreSQL con work_mem = 64MB	MongoDB (allowDiskUse: true)
Tiempo de Ejecución Real	**19,084.40 ms** (~19.1 s)	**3,737.23 ms** (~3.7 s) [**5.1× más rápido**]	**26,605.00 ms** (~26.6 s)
Método de Escaneo	Parallel Bitmap Heap Scan (Lossy)	Parallel Bitmap Heap Scan (Exact)	COLLSCAN + In-Memory Sort Buffer
Heap Blocks Examined	Exact=6,846 | **Lossy=10,728**	**Exact=13,511 | Lossy=0**	249,802 docs examinados
Buffers de I/O	Shared Hit=3,446 | **Read=50,744**	Shared Hit=48 | **Read=42,093**	WiredTiger Cache + Disk Spill
Método de Ordenamiento	Top-N Heapsort (Memoria: 277 kB)	Top-N Heapsort (Memoria: 274 kB)	Blocking Sort (allowDiskUse)
Causa Raíz de Desempeño	Al agotar 4MB, el mapa de bits se degrada a nivel página (lossy), forzando re-lectura y re-chequeo de tuplas en disco.	64MB permite almacenar punteros exactos a tuplas en memoria, evitando lecturas redundantes.	Filtra y ordena mediante pipeline canalizado, penalizado por el costo de serialización BSON.

6. Interpretación y Respuestas a Preguntas de Investigación

M1: ¿Cómo cambian la latencia P95 y P99 de las agregaciones sobre documentos anidados cuando se limita de forma estricta la CPU y la memoria?

Respuesta Empírica: La limitación estricta de CPU (CFS quota) introduce un factor de amplificación de latencia no lineal que castiga desproporcionadamente a las consultas con operaciones de desanidamiento en memoria. En **MongoDB**, la latencia P95 de Q1 a C=1 pasa de 1,554 ms en Baseline (4 CPUs) a 1,751 ms en Severe (1 CPU), pero a C=50 clientes, la latencia P95 escala de 157,906 ms a 232,696 ms con un estrangulamiento de CPU acumulado de **4,872 segundos**. A pesar de este retraso severo, MongoDB mantuvo un **comportamiento elástico y 100% libre de fallas**.
En contraste, **PostgreSQL JSONB** demostró incapacidad de tolerar alta concurrencia (C=50) en Q1 bajo cualquier perfil, sufriendo desconexiones forzadas y colapso de hilos clientes debido al costo cuadrático de desempaquetar arreglos JSONB en memoria mediante jsonb_array_elements sin paralelismo cooperativo.

M2: ¿Qué motor usa de forma más eficiente los buffers de ordenamiento y agrupamiento antes de volcar a disco temporal o fallar por falta de memoria (OOM)?

Respuesta Empírica: La eficiencia depende de la naturaleza de la consulta. En operaciones de **ordenamiento Top-K (Q2)**, **PostgreSQL demostró una eficiencia algorítmica superior**: al contar con work_mem = 64MB, su algoritmo de *Top-N Heapsort* operó consumiendo únicamente **274 kB de memoria de ordenamiento** para producir el Top-1000 en 3.73 segundos, superando ampliamente a MongoDB. Sin embargo, cuando work_mem se limitó a 4MB, se registraron **27 eventos de volcado a disco/pérdida de resolución de bitmap** (disk_spill = True), degradando su tiempo a 19.08 segundos.
Por su parte, **MongoDB** gestionó los buffers de ordenamiento y volcado a disco de manera **completamente autónoma y transparente** mediante el flag allowDiskUse: true, sin registrar fallas de proceso ni desbordamientos incontrolados, amortiguando la falta de memoria física a costa de latencia adicional.

M3: ¿Cuál es la huella de CPU y memoria por cada 1,000 documentos procesados en el motor documental frente al motor con JSONB?

Respuesta Empírica: Con un volumen corpus de 500,000 documentos indexados:
• **MongoDB:** Registró una huella de memoria residente (RSS) constante oscilando entre **448 MB y 899 MB** en total, lo cual representa un consumo específico de **0.89 MB a 1.79 MB de RSS por cada 1,000 documentos**. En CPU, el motor documental demandó menos ciclos computacionales por documento en agregaciones complejas (Q1), ya que el formato binario BSON pre-indexa el tamaño de campos y offsets, permitiendo recorrer arreglos anidados sin re-parsear cadenas de texto.
• **PostgreSQL JSONB:** Exhibe una huella de proceso base más pequeña (1.1 MB a 3.6 MB de RSS visible por backend individual), apalancada en el bloque maestro de shared_buffers. No obstante, en consumo de CPU por cada 1,000 documentos, PostgreSQL requiere un costo significativamente mayor en Q1, pues jsonb_array_elements demanda des-serialización y casteo explícito de tipos (::numeric, ->>) en tiempo de ejecución para cada elemento del arreglo.

7. Recomendaciones Técnicas para Despliegues en la Nube

Dimensión de Arquitectura	Criterio y Decisión de Ingeniería	Recomendación Práctica para Producción
Selección del Motor	MongoDB para analítica nativa sobre documentos anidados; PostgreSQL para cargas híbridas transaccionales.	Si la carga primaria involucra $unwind de arreglos a escala, desplegar MongoDB. Si se requieren ACID estricto y JOINs relacionales con datos tabulares, PostgreSQL JSONB.
Sizing de Recursos (Cloud)	Dimensionamiento de instancias en GCP, AWS o Azure.	Mínimo 4 vCPUs y 8 GB RAM para producción. Evitar perfiles de 1 vCPU / 1 GB con agregaciones masivas para prevenir estrangulamiento de CFS > 1000s.
Tuneo de Memoria en PostgreSQL	Configuración crítica de work_mem para evitar pérdida de resolución bitmap.	Establecer ALTER SYSTEM SET work_mem = '64MB'; global, y permitir hasta 256MB por sesión para reportes analíticos pesados.
Tuneo de Agregación en MongoDB	Garantizar ejecución elástica sin errores de tope de 100MB.	Siempre incluir { allowDiskUse: true } en pipelines de analítica y ampliar internalQueryMaxBlockingSortMemoryUsageBytes a 256MB.
Estrategia de Indexación	Mitigación de escaneos de colección completos (COLLSCAN).	En PostgreSQL: Índices funcionales B-Tree sobre expresiones JSONB extraídas frecuentemente (timestamp, total). En MongoDB: Índices compuestos en payment.status + timestamp.

Políticas de Alerta (Prometheus)	Telemetría continua de contención y umbrales SLA.	Disparar alerta crítica si container_cpu_cfs_throttled_seconds_total supera 10s por minuto o si RSS excede el 85% de la cuota asignada.

Apéndice: Especificación de Entregables y Repositorio

Cumplimiento de Entregables del Playbook (§8):

• **Repositorio Git (§8.1):** Contiene el código fuente parametrizado (docker-compose.yml), el generador sintético de 500k registros (synthetic_data_generator.py), el runner multiproceso con drivers nativos (benchmark_runner.py), las consultas estructuradas en queries/, y el script de automatización run_benchmarks.sh / run_strict.py.

• **Matriz CSV (§8.2):** Archivo results/benchmark_results.csv con 162 registros y las 14 columnas requeridas (*timestamp_epoch, engine, profile, query_id, concurrency, run_number, throughput_qps, latency_p50_ms, latency_p95_ms, latency_p99_ms, cpu_throttled_seconds, max_rss_mb, disk_spill, status*).

• **Reporte Formal (§8.3):** El presente documento compila íntegramente las 7 secciones obligatorias con gráficos empíricos, tablas consolidadas, análisis de planes de ejecución y recomendaciones técnicas.